



INFORMATICS
INSTITUTE OF
TECHNOLOGY

Design & Testing Document

For W2212841_recipe_manager.py
(4COSC006C: Software Development I)

Yahya (IIT – 20241597) (UOW – W2212841)

Abstract

This document shows the algorithms, design ideas and decisions made to create a Python CLI based application that manages recipes. This application is `W2212841_recipe_manager.py` and was created by N.M.Yahya (IIT ID: 20241597, UOW ID: W2212841). Furthermore, this document also contains test cases that prove the functionality and capabilities of the program and is concluded with a reflection indicating the experience of the programmer.

Acknowledgements

I would like to first acknowledge God and the opportunity he has granted me to be able to study at IIT.

I would then like to show my appreciation to the module leader and lecturer, Mr. Guhanathan Poravi, as well as the tutorial lecturers Mr. Nevindu Jayathilake and Ms. Abishani Muthalagan, for their dedication to imparting the knowledge necessary for me to complete this coursework successfully.

I would also like to thank my family and friends for their love and support.

I would also like to acknowledge the other staff at IIT and the resources provided by the institute to successfully complete this coursework.

Finally, I would like to show appreciation to the many free online resources that made it possible to self-learn and clarify any doubts about completing this project.

Table of Contents

Abstract	i
Acknowledgements	ii
Table of Contents	iii
List of Tables	v
List of Abbreviations	vi
Introduction	1
Design	2
Main Algorithm.....	3
Add Recipe.....	4
Edit Recipe.....	4
Duplicate Recipe	5
View Recipe	5
Export Recipe.....	6
Delete Recipe	7
View Statistics.....	7
Exit.....	7
Other Algorithms	8
Validation.....	8
Prompt.....	8
Save and Read	9
Display	10
Tasks.....	10
Function Decomposition	10
Justifications	11
Data Structures	11
File Handling	12
Test Cases.....	13
Reflection	14
Conclusion	15

List of Figures

Figure 1 - Structure of Directories for the Recipe Manager	2
Figure 2 - Flowchart for the Main Match:Case Structure	3
Figure 3 - Nested Dictionary.....	11

List of Tables

Table 1 - Justification for Data Structures	12
Table 2 - Test Case Legend	13
Table 3 - Test Cases	13

List of Abbreviations

AI – Artificial Intelligence

cmd – Command Prompt

DRY – Don't Repeat Yourself

ID – Identity

IIT – Informatics Institute of Technology

JSON – JavaScript Object Notation

UOW – University of Westminster

YAML – YAML Ain't Markup Language

Introduction

The coursework asks students to create a “Recipe Book Manager” in the Python language that will run in command line (cmd). This recipe book manager must be able to add recipes, edit recipes, duplicate recipes, delete recipes, save recipes to a file, load recipes from a file, export recipes in a readable format and view statistics of the recipes while handling files, displaying graceful messages, providing an intuitive experience and prevent users from saving invalid files.

To accomplish this, the appropriate use of data structures, algorithms and use of internal libraries was used. In addition to executing the program from a main file, several other packages (directories) containing modules (.py files) and functions were created and used. This helps the code be more reusable (following DRY principles), more scalable and easier to point issues. The modularization also makes it easier to understand which sort of functions can be found in which sort of modules. For example: a function to read data from a file will not be found in a “save” module and vice versa.

Throughout this document, the approaches taken to tackle all these issues and create a program that is of an excellent quality will be looked at.

Design

Before jumping right into the code, and explaining the algorithmic approaches used, it has been decided that it is better to first explain the directories and files used, so that it will be easier to refer to later and explain what is happening.

Below an image will be given indicating the structure of the directories and the files within them, furthermore a short description will be given for each file along with each of the individual functions within the modules (if the files are modules) so that they can be referred to and referenced when explaining a particular design choice.

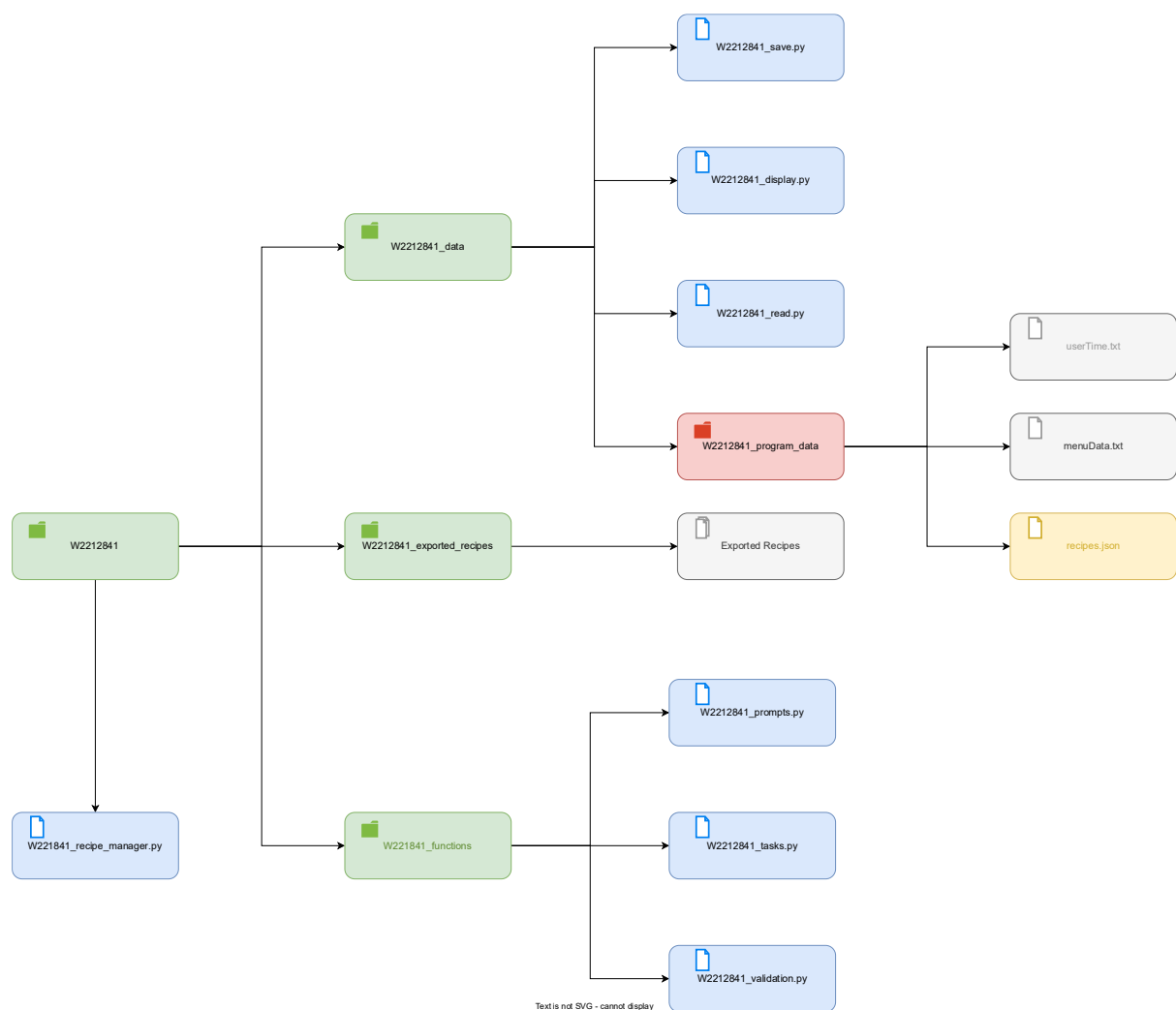


Figure 1 - Structure of Directories for the Recipe Manager

The main folder holding everything is W2212841 (named after the UOW ID), within which is the main program (W2212841_recipe_manager.py). This is the python file which is executed to access the recipe book manager. Within main folder are three directories:

1. W2212841_data: this directory stores the modules and another directory (W2212841_program_data) which are related to the data needed to allow the program function.
2. W2212841_exported_recipes: this directory stores the recipes that exported as text files
3. W2212841_functions: this directory stores the modules store the functions that perform the grunt work including validation of user entries, prompting the user for choices and the main tasks of the programme.

Main Algorithm

The main algorithm of the project is a **nested match:case (switch:case) structure** within a while loop. This **while loop** is necessary to ensure that the user continues to stay within the main menu of the program and that the user can return to the main menu whether they complete or cancel a task they begin. Within the **while loop** we also validate the choice (input) and see **if** it is correct, this allows us to stop the user from entering the **match:case selection structure** with an invalid value for choice. We will also look in depth at how the user inputs get validated when we look deeply at the algorithmic structure of W2212841_validation.py later.

This can be simply shown by this pseudocode and the flow chart afterwards:

```
menu ← True
```

```
WHILE menu = True:
```

```
    INPUT AND VALIDATE choice
```

```
    MATCH choice:
```

```
        // See Figure below for detailed flowchart
```

```
END WHILE
```

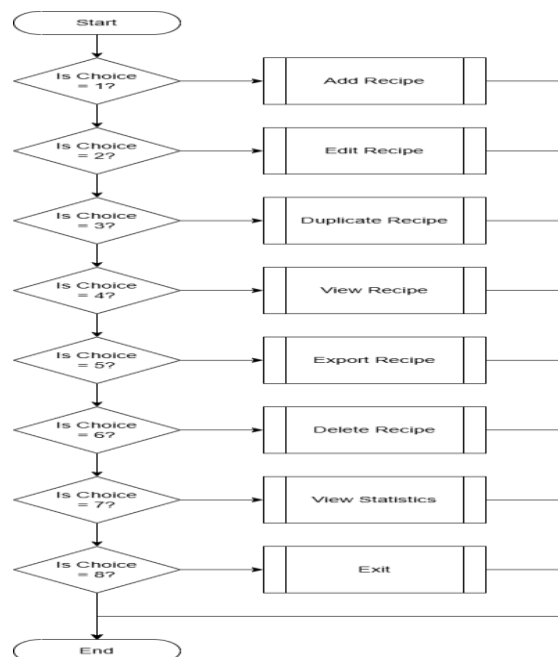


Figure 2 - Flowchart for the Main Match:Case Structure

Here it can be seen that every choice leads to its own named process, while these subroutines are not actual subroutines that are performed regularly across the program in different areas, their function is persistent across every instance that the program is loaded. We will now look at the pseudocode for every subroutine and explain the algorithms used for each named process.

Add Recipe

```
FUNCTION AddRecipe:
    INPUT AND VALIDATE recipe
    SAVE recipe to file
END FUNCTION
```

In this named process, the program adds a new recipe by means of the function addRecipe() from W2212841_tasks.py module. The details about the recipe include the name of the recipe, ingredients of the recipe, time taken for the recipe, category of the recipe and the tags of the recipe are all validated using validation functions present in the W2212841_validation.py module. Since the validation for almost all functions works in a similar algorithm, we will investigate how these functions work later.

Other than the validation, this function then prompts the user as to whether they want to save the recipe, the user can **decide** yes or no, and if the user **selects** yes then the recipe is saved automatically, **else** the recipe is not saved, this is achieved with the help of the saveRecipe() function from W2212841_save.py module. The function finally asks **whether** the user wants to add another recipe, and if so loops again. Here a similar **while loop** is used where the state of adding another recipe is assigned as **True** and continues to be **True** until the user states otherwise.

Edit Recipe

```
FUNCTION EditRecipe:
    IF recipe list is empty THEN
        DISPLAY error message
        RETURN
    END IF
    INPUT AND VALIDATE recipe selection
    DISPLAY recipe
    INPUT AND VALIDATE new value
    SAVE updated recipe to file
END FUNCTION
```

Ironically, the program calls the same addRecipe() function. While this is tedious for the user as it forces the user to enter every value again, it allows users who want to edit multiple aspects of their recipe the freedom to do so. Given the choice between freedom and convenience, the testers preferred freedom and it was chosen as the avenue to follow.

This named process, however, differs by displaying the selected recipe and its details and allows users to view the existing recipe while making their changes to it.

Finally, this named process calls the saveRecipe() function once more to save the newly edited recipe.

Duplicate Recipe

```
FUNCTION DuplicateRecipe:
    IF recipe list is empty THEN
        DISPLAY error message
        RETURN
    END IF
    INPUT AND VALIDATE recipe selection
    COPY selected recipe
    SAVE duplicated recipe to file
END FUNCTION
```

In this named process, the program duplicates existing recipes. While initially it was decided to force the user to edit the recipe once it was duplicated, it was discovered that there were multiple ways by which a user could bypass this ruling and simply duplicate the recipe multiple times without changing a single attribute. This discovery lead to the conclusion that users should be free to duplicate a recipe as many times as they want with zero changes, and if users felt the need to change it themselves they have the tools to do so.

This named process calls the duplicateRecipe() function from W2212841_tasks.py module to duplicate the recipe. Finally, it also calls the saveRecipe() function from W2212841_save to save the recipe to the save file.

View Recipe

```
FUNCTION ViewRecipe:
    INPUT AND VALIDATE choice
    MATCH choice:
        CASE 1: DISPLAY all recipes
        CASE 2: INPUT selection → DISPLAY specific recipe
        CASE 3: INPUT filters → DISPLAY filtered recipes
```

CASE 4: INPUT search term → DISPLAY matching recipes

END FUNCTION

This named process opens into 4 other named processes via a **match:case selection structure** (if the same logic that is applied for the above-named processes applies here as well), however simply explaining the functions called and what they do is easier.

For case 1, the named process calls the multipleRecipes() function from the W2212841_display.py module. This function was **AI GENERATED** using Claude AI (Sonnet 4.6), the argument for the acceptability of this code is that the code only has a visual output which enhances the user experience and does not affect the core functionality. This function simply looks at the recipes (which are saved in a dictionary) and displays them in a tabular manner.

For case 2, the named process calls the viewRecipe() function from the W2212841_tasks.py module. This is because viewRecipe() is a function that validates the user input recipe ID [promptID()], prompts the user **if** they want to view another recipe or not [addingValidation()], and displays the recipe[recipe()] using functions from various modules. This occurs only **if** there are any recipes present in the recipe dictionary.

For case 3, the named process calls the filterRecipe() function which filters the recipes based on the users' choices and displays them using the filteredRecipes() function. These functions are from the W2212841_tasks.py and W2212841_display.py modules respectively.

For case 4, the named process calls the searchRecipe() function from the W2212841_tasks.py module. This is because like viewRecipe() earlier, it is composed of multiple functions across multiple modules.

Export Recipe

FUNCTION ExportRecipe:

```
    IF recipe list is empty THEN
        DISPLAY error message
        RETURN
    END IF
    INPUT AND VALIDATE recipe selection
    WRITE recipe to external file
```

END FUNCTION

This named process performs the exportRecipe() function from the W2212841_tasks.py module. This function validates the user input recipe ID, confirms **whether** the user wants to validate multiple recipes or not, and **whether** the user wants to export the multiple recipes as individual files or as a single file.

Delete Recipe

```
FUNCTION DeleteRecipe:
```

```
    INPUT AND VALIDATE choice
```

```
    MATCH choice:
```

```
        CASE 1: INPUT selection → DELETE selected recipe from  
file
```

```
        CASE 2: REQUEST confirmation → DELETE all recipes from  
file
```

```
END FUNCTION
```

This named process is like [View Recipe](#) from earlier, in that it leads to multiple smaller named processes using a match:case selection structure. Here, when the user **decides** to delete a single file (Case 1) the process calls the `delRecipe()` function from the `W2212841_tasks.py` module. This function validates the user input, collects user confirmation and deletes the selected recipe from the recipe dictionary. This function only allows the user to prompt to delete when there are any recipes present in the recipe dictionary.

If the user decides to delete all the recipes (Case 2), the process calls the `remove()` function from the `os` module (`os` is a python library) and then deletes the entire save file. This is an easy approach to the problem since, **if** the program detects no save file then it will create a new save file and start from scratch.

View Statistics

```
FUNCTION ViewStatistics:
```

```
    DISPLAY statistics
```

```
END FUNCTION
```

This process calls the `allStatistics()` function from the `W2212841_display.py` module and then displays all the necessary statistics. While this function may simply display all the necessary information, it works with other functions to calculate the necessary values.

Exit

```
FUNCTION Exit:
```

```
    SAVE session time
```

```
    DISPLAY exit message
```

```
    TERMINATE program
```

```
END FUNCTION
```

This process simply terminates the entire program. The save session time refers to the function `save.saveTime()` which allows the program to calculate how long the user has been running the program, while not a necessary statistic to display, it's always a very fun statistic to know.

This process also displays an exit message using a function `exitMessage()` from `W2212841_display.py` module before using the built-in `exit()` function to close the application.

Other Algorithms

In this part of the document, we shall now focus on certain functions and modules to see what other algorithms have been used to achieve the final goal of the functioning recipe manager.

Validation

First, the validation functions of the `W2212841_validation.py` module shall be discussed, as we have seen that across the main algorithm they have been used and mentioned in several different functions across different modules.

This module contains five functions:

1. `nameValidation()` – Used to validate the names of recipes, ingredients and tags
2. `addingValidation()` – Used to validate the choice of a user (initially it was used to validate specifically if a user wanted to add another recipe, however its functionality was applicable across a broad spectrum of use case scenarios)
3. `timeValidation()` – Used to validate the time taken of a recipe
4. `IDValidation()` – Used to validate the recipe ID in the recipe dictionary
5. `searchValidation()` – Used to validate the searchword of a user to search for specific recipes

All these modules function in a variation of the following pseudocode:

```
FUNCTION Validate(input) :  
    CHECK input against rules  
    RETURN True or False  
END FUNCTION
```

The only exception to this is the `addingValidation()` as it also reprompts the user **until** the user inputs a valid answer. This functions in a similar way to the loop that occurs in the main algorithm using a **while loop** that is broken only when the user inputs a valid input.

Prompt

Throughout the program, the program prompts the user for inputs, these include inputs that are used in other functions such as `addRecipe()`, `editRecipe()`, etc. Every prompting function uses an infinite while loop that is only broken once a valid input is input. The prompting functions are only able to figure out which inputs are valid using the validation functions in the

W2212841_validation.py module. In fact, only one function in this module, W2212841_prompts.py, doesn't make use of a validation function.

This module also has five functions:

1. promptName() – Prompts for names used in recipes, ingredients and tags
2. promptMenu() – Prompts the user for an input based on the menu they see, this function doesn't validate using a validation function and instead validates based on the indexes of the list it uses (Menus are stored as lists when the program is running)
3. promptTime() – Prompts for the time taken for a recipe
4. promptID() – Prompts for the ID of a recipe
5. promptSearch() - Prompts for the searchword that will be used to search for specific recipes

A very clever way of warning the user about incorrect prompts is by using a count a variable that iterates with the **while loop**, the count starts at 0, and the function looks at bool(count) to see if the user has typed in something wrong, because bool(x) where x is < 0 will always return True.

The common pseudocode in these functions is given as follows:

```
FUNCTION Prompt(parameters):
```

```
    WHILE input is invalid:
```

```
        DISPLAY rules
```

```
        INPUT value
```

```
        VALIDATE value
```

```
    RETURN value
```

```
END FUNCTION
```

Save and Read

There are 10 functions spread across two modules (W2212841_save.py and W2212841_Read.py) that deal with file handling. These functions are also nothing special and try to read or write data into text files (or .json file in the case of recipes) and have failsafes to prevent the program from halting just because of corrupted or missing data.

The common pseudocode for the functions across these modules is:

```
FUNCTION ReadOrSave(file):
```

```
    IF file exists:
```

```
        OPEN file
```

```
        READ / WRITE data
```

```
    ELSE:
```



```
        CREATE file
        READ / WRITE default data

END FUNCTION
```

The only exception would be the `readRecipesCorrect()` function, which converts the data flattened due to dumping it into a .json back into usable data.

Display

This is another module that is packed with function. The module `W2212841_display.py` has 17 functions, with one function even having a nested function across 252 lines. Most of this are obviously lines of `print()` which simply contain the information that needs to be displayed.

The functions in this module follow these lines of pseudocode:

```
FUNCTION Display(data) :
    PROCESS data
    DISPLAY output
END FUNCTION
```

Some functions in this module do some logic however, for example the `searchedRecipes()` does some logic to display at least partial matching inputs.

Tasks

Throughout the report, the module `W2212841_tasks.py` and its functions have been mentioned. The only similarity between all these functions is that they accomplish a task relevant to the program. Going through the algorithms of each function here also seems pointlessly exhausting given that these functions are the most complicated, calling in sometimes more than 5 functions across the various modules just to finish their task, and there are more than 10 functions spread across over 240 lines of code.

Function Decomposition

The previous sections [Main Algorithm](#) and [Other Algorithms](#), indirectly show how the algorithms function utilizing functions across all modules repeatedly; a clear indication of use of the DRY principle.

Furthermore, functions were also decomposed based on their use case. This allows for scalability and ease of navigation, after looking through this codebase and document. It makes sense that a new function, perhaps a function that displays the images of the recipes will be stored in `W2212841_display.py` and not `W2212841_tasks.py`.

Functions across the project were maintained to be either 30 lines or less (most of the functions are under 30 lines, those that are 30 lines are large only due to the blank lines in between to

make the code more readable and including comments), and function decomposition helped to enforce DRY principles, scalability and navigability.

Justifications

Data Structures

For this project the most sensible data structure to store the details about recipes was deemed a nested dictionary, where the main dictionary stored all the recipes as dictionaries that were the values to the keys inside the main dictionary. Those nested dictionaries, as dictionaries, could also be **iterated** through allowing the program to easily manipulate, read and store the necessary information required.

This can all be easily explained diagrammatically:

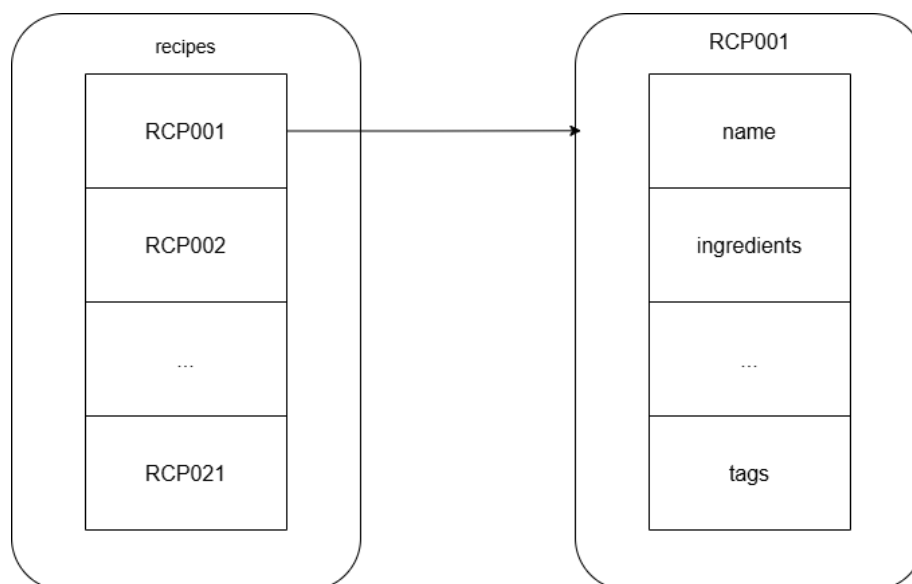


Figure 3 - Nested Dictionary

As seen in the above diagram, when a recipe is accessed we can then access its relevant information by accessing its keys and returning it.

Now that the argument for the use of a nested dictionary has been presented, let's look at the data structures used for the other categories and the arguments that can be made to justify the usage of those data structures.

Stored Information	Data Structure Used	Argument
Ingredients	Tuple	Ingredient name, value and measuring quantity will not change.
Tags	Set	Prevents the addition of duplicate tags

Table 1 - Justification for Data Structures

File Handling

In this project, two types of files are used:

1. JSON files
2. .txt files

The usage of .txt files is easily justifiable given that the exported recipes must be parsable by the average user, the .txt format also allows users to transfer the media to other file types including .md, .docx and .pdf file types.

As for the usage of the JSON file, it is not completely justifiable given that Python structures like sets and tuples get flattened into arrays when dumped into a .json file, in fact sets cannot be dumped on their own and must be converted first into a list to be dumped. A more acceptable file type would have been the pickle file type, however since the specification for this project prevents the use of external libraries, the second-best choice was to use JSON.

Another argument against the JSON file is that while it is easily loaded and saved into and from the Python file it is not the most readable, in fact a YAML file would have been a better choice, however once again due to specification limitations, the second-best choice was to use JSON.

Despite these valid arguments, the JSON file still remained the most convenient method of saving the recipes data and was therefore selected.

Test Cases

To prove the functionality of the program, an extensive list of test cases has been prepared including regular use cases, irregular use cases and edge cases to stress the limits of the program and see if it is a quality product.

Legend:
Regular/Valid
Irregular/Invalid
Edge Case

Table 2 - Test Case Legend

Test Case ID	Function Tested	Case Type	Input / Process	Expected Outcome	Actual Outcome	Pass/Fail
1	Application Startup	Regular/Valid	Run the program without	Successful message displayed, recipe count shown above the recipe list	Successful message displayed, recipe count shown above the recipe list	Pass
2	Application Startup	Edge Case	Run the program with no existing recipes.json	File not found, 10 recipes loaded from the database, program starts normally	File not found, 10 recipes loaded from the database, program starts normally	Pass
3	Application Startup	Irregular/Invalid	Run with a corrupted recipes.json	File loaded, new data file created, program starts with 3 recipes	File loaded, new data file created, program starts with 3 recipes	Pass
4	Main Menu	Regular/Valid	Enter a valid option number (e.g. 1)	Navigates to the corresponding feature	Navigates to the corresponding feature	Pass
5	Main Menu	Irregular/Invalid	Enter a value (e.g. 0)	Input rejected, main menu re-displayed	Input rejected, main menu re-displayed	Pass
6	Main Menu	Irregular/Invalid	Enter a number out of range (e.g. 9)	Input rejected, main menu re-displayed	Input rejected, main menu re-displayed	Pass
7	Main Menu	Edge Case	Enter 0	Input rejected, main menu re-displayed	Input rejected, main menu re-displayed	Pass
8	Add Recipe	Regular/Valid	Enter option 1 → enter valid recipe ID, ingredients, add time, category, no tags → confirm "Y"	Recipe saved, full-screening recipe message shown	Recipe saved, full-screening recipe message shown	Pass
9	Add Recipe	Regular/Valid	Complete a valid recipe → confirm save → answer "Y" to add another	First recipe saved, add recipe flow restarted	First recipe saved, add recipe flow restarted	Pass
10	Add Recipe	Regular/Valid	Complete a valid recipe → confirm save → answer "Y" to add another	Recipe saved, returns to main menu	Recipe saved, returns to main menu	Pass
11	Add Recipe	Regular/Valid	Complete a valid recipe → decline save with "N"	Recipe discarded, not written to file	Recipe discarded, not written to file	Pass
12	Add Recipe	Irregular/Invalid	Enter a recipe name shorter than 3 characters (e.g. "a")	Name rejected, re-prompted with naming criteria	Name rejected, re-prompted with naming criteria	Pass
13	Add Recipe	Irregular/Invalid	Enter a recipe name with an invalid special character (e.g. "a@")	Name rejected, re-prompted with naming criteria	Name rejected, re-prompted with naming criteria	Pass
14	Add Recipe	Irregular/Invalid	Enter a name that is only digits (e.g. "12345")	Name rejected, re-prompted	Name rejected, re-prompted	Pass
15	Add Recipe	Irregular/Invalid	Enter ingredient quantity as 0 or negative	Quantity rejected, re-prompted for a numeric value	Quantity rejected, re-prompted for a numeric value	Pass
16	Add Recipe	Irregular/Invalid	Enter ingredient quantity as 0 or negative	Quantity rejected, re-prompted	Quantity rejected, re-prompted	Pass
17	Add Recipe	Irregular/Invalid	Enter preparation time in wrong format (e.g. "130" or "1:30")	Time rejected, re-prompted with time criteria	Time rejected, re-prompted with time criteria	Pass
18	Add Recipe	Irregular/Invalid	Enter preparation time outside allowed range (e.g. "10:00" or "15:00")	Time rejected, re-prompted	Time rejected, re-prompted	Pass
19	Add Recipe	Edge Case	Add a recipe with exactly 3 ingredients	Recipe accepted and saved normally	Recipe accepted and saved normally	Pass
20	Add Recipe	Edge Case	Add a recipe with 20 ingredients (maximum)	All 20 ingredients accepted, no 21st ingredient prompted	All 20 ingredients accepted, no 21st ingredient prompted	Pass
21	Add Recipe	Edge Case	Enter a recipe name of exactly 50 characters	Name accepted	Name accepted	Pass
22	Add Recipe	Edge Case	Add a tag that isn't another tag to the same recipe	Both tags stored in the recipe's tag set	Both tags stored in the recipe's tag set	Pass
23	Edit Recipe	Regular/Valid	Select option 2 → enter valid ID → confirm edit "Y"	Old recipe replaced with new data, JCSN updated, success message shown	Old recipe replaced with new data, JCSN updated, success message shown	Pass
24	Edit Recipe	Regular/Valid	Select option 2 → enter valid ID → decline edit with "N"	Recipe unchanged, returns to main menu	Recipe unchanged, returns to main menu	Pass
25	Edit Recipe	Irregular/Invalid	Enter an ID that does not exist (e.g. "999999")	ID rejected, re-prompted	ID rejected, re-prompted	Pass
26	Edit Recipe	Irregular/Invalid	Enter an ID in wrong format (e.g. "ABCDEF")	ID rejected, re-prompted	ID rejected, re-prompted	Pass
27	Duplicate Recipe	Regular/Valid	Select option 3 → enter valid ID → confirm duplicate "Y"	New recipe created with new date, JCSN updated, success message shown	New recipe created with new date, JCSN updated, success message shown	Pass
28	Duplicate Recipe	Regular/Valid	Select option 3 → enter valid ID → decline with "N"	No duplicate created, returns to main menu	No duplicate created, returns to main menu	Pass
29	Duplicate Recipe	Irregular/Invalid	Enter an ID that does not exist	ID rejected, re-prompted	ID rejected, re-prompted	Pass
30	View → View All Recipes	Regular/Valid	Select option 4 → 1 → with no recipes in the book	Table displayed with all recipes, columns correctly aligned	Table displayed with all recipes, columns correctly aligned	Pass
31	View → View All Recipes	Edge Case	Select option 4 → 1 → with no recipes saved	"No recipes found" displayed	"No recipes found" displayed	Pass
32	View → View Specific Recipe	Regular/Valid	Select option 4 → 2 → enter valid ID	Full recipe details displayed (name, category, time, tags, ingredients)	Full recipe details displayed (name, category, time, tags, ingredients)	Pass
33	View → View Specific Recipe	Regular/Valid	After viewing, answer "Y" to see another	Prompted for another recipe ID	Prompted for another recipe ID	Pass
34	View → View Specific Recipe	Regular/Valid	After viewing, answer "N"	Returns to main menu	Returns to main menu	Pass
35	View → View Specific Recipe	Edge Case	View a recipe that has no tags	Tags: None! shown correctly	Tags: None! shown correctly	Pass
36	View → View Specific Recipe	Irregular/Invalid	Enter non-existent ID	ID rejected, re-prompted	ID rejected, re-prompted	Pass
37	View → Filter Recipes	Regular/Valid	Filter by category: DINNER only	Only DINNER recipes shown in table	Only DINNER recipes shown in table	Pass
38	View → Filter Recipes	Regular/Valid	Filter by time range only (e.g. 00:00 to 00:05)	Only recipes within time range shown	Only recipes within time range shown	Pass
39	View → Filter Recipes	Regular/Valid	Filter by ingredient count only (e.g. 3 to 5)	Only recipes with 3-5 ingredients shown	Only recipes with 3-5 ingredients shown	Pass
40	View → Filter Recipes	Regular/Valid	Apply all three filters together	Only recipe matching all three criteria shown	Only recipe matching all three criteria shown	Pass
41	View → Filter Recipes	Edge Case	Apply all filters (answer "N" to all)	Full recipe list displayed	Full recipe list displayed	Pass
42	View → Filter Recipes	Edge Case	Apply a filter that no recipe matches	"No recipes found" displayed	"No recipes found" displayed	Pass
43	View → Search	Regular/Valid	Search for a word found in a recipe name	Matching recipe(s) shown in table	Matching recipe(s) shown in table	Pass
44	View → Search	Regular/Valid	Search for a word found in an ingredient	Recipe containing that ingredient shown	Recipe containing that ingredient shown	Pass
45	View → Search	Regular/Valid	Search for a word found in a tag	Recipe with that tag shown	Recipe with that tag shown	Pass
46	View → Search	Regular/Valid	Search using uppercase (e.g. "CURRY")	Case insensitive search returned	Case insensitive search returned	Pass
47	View → Search	Edge Case	Search for a word with no matches	Empty table or "No recipes found" displayed	Empty table or "No recipes found" displayed	Pass
48	View → Search	Irregular/Invalid	Enter a searchword shorter than 3 characters	Search rejected, re-prompted	Search rejected, re-prompted	Pass
49	Export → Single Recipe	Regular/Valid	Select option 5 → enter 1 valid ID → decline exporting another	File is created if requested, recipe folder with correct recipe details	File is created if requested, recipe folder with correct recipe details	Pass
50	Export → Multiple (Separate)	Regular/Valid	Select option 5 → select 2 valid IDs → choose "Multiple" first	2 separate .txt files created, one per recipe	2 separate .txt files created, one per recipe	Pass
51	Export → Multiple (Combined)	Regular/Valid	Select option 5 → select 2 valid IDs → choose "A Single File" → enter valid filename	1 combined .txt file created with both recipes	1 combined .txt file created with both recipes	Pass
52	Export → Multiple (Combined)	Irregular/Invalid	Enter a filename that already exists	Warning shown, re-prompted for a different filename	Warning shown, re-prompted for a different filename	Pass
53	Export	Irregular/Invalid	Select the same recipe ID twice for export	Recipe already selected warning shown	Recipe already selected warning shown	Pass
54	Delete → Specific Recipe	Regular/Valid	Select option 6 → 1 → valid ID → confirm "Y"	Recipe removed from book, JCSN updated, success message shown	Recipe removed from book, JCSN updated, success message shown	Pass
55	Delete → Specific Recipe	Regular/Valid	Select valid ID → decline delete with "N"	Recipe unchanged	Recipe unchanged	Pass
56	Delete → Specific Recipe	Regular/Valid	After deleting, answer "Y" to delete another	Prompted for another ID	Prompted for another ID	Pass
57	Delete → Specific Recipe	Irregular/Invalid	Enter non-existent ID	ID rejected, re-prompted	ID rejected, re-prompted	Pass
58	Delete → Specific Recipe	Edge Case	Delete the only remaining recipe	Recipe book becomes empty, JCSN updated	Recipe book becomes empty, JCSN updated	Pass
59	Delete → Delete All	Regular/Valid	Select option 6 → 2 → confirm with "Y"	No recipes remain, success message shown	No recipes remain, success message shown	Pass
60	Delete → Delete All	Regular/Valid	Select option 6 → 2 → decline with "N"	No recipes deleted, returns to main menu	No recipes deleted, returns to main menu	Pass
61	Delete → Delete All	Edge Case	Confirm delete all when only 1 recipe exists	Recipe book emptied, JCSN removed	Recipe book emptied, JCSN removed	Pass
62	View Statistics	Regular/Valid	Select option 7 with multiple recipes saved	Full recipe statistics displayed: tag, ingredients, average ingredients, season file, all time totals shown	Full recipe statistics displayed: tag, ingredients, average ingredients, season file, all time totals shown	Pass
63	View Statistics	Edge Case	Select option 7 with no recipes saved	Text: 0, categories all show 0, program does not crash	Text: 0, categories all show 0, program does not crash	Pass
64	Quit	Regular/Valid	Select option 8	Quit message displayed, session time saved to userTime.txt program closes	Quit message displayed, session time saved to userTime.txt program closes	Pass
65	Quit	Edge Case	Quit on first save (you saveTime.txt exists)	UserTime.txt created, session time written, program closes	UserTime.txt created, session time written, program closes	Pass

Table 3 - Test Cases

Reflection

By building this application I was able to learn a lot. This project allowed me to refresh concepts such as data structures, algorithms, file handling, and modularity of functions.

I also learned new things such as handling .json files, which gave me exposure to other formats like YAML and pickle. While I had theoretical knowledge of sets in Python, this project gave me the opportunity to solidify that understanding through hands-on experience.

One limitation I identified was my menu storage system. Although storing menus in an external text file made them easy to visualise and edit, relying on it was a vulnerability. My current fix is a band-aid solution at best. Going forward, I would like to create a function that automatically generates match:case selection logic, so that the programmer only needs to edit the text file and the rest is handled automatically.

I also enjoyed learning how AI can assist in development — writing trivial code, organising thoughts, bugfixing, and generating secondary documents (the sample .json, test cases table and even this summarised reflection; from 370 words to 251, was AI-generated).

On a less technical note, this project taught me a valuable lesson in time management. When I followed my schedule and dedicated time properly, I was well ahead and able to build the solution at a methodical pace. However, towards the end of the semester, my available time shrank while my task backlog did not.

All in all, it was a wonderful and stressful experience — one I wouldn't mind repeating

Conclusion

This document provides valid justification to the quality of the code provided demonstrated through the explanation of the algorithmic design, data structures used, file types used and test cases tested.

A reflection of the owner is also present which shows the personal growth of the developer.